



version 1.2.2

User guide and reference

Mico Mrkaic

July 11, 2026

Abstract. `tea` is a small command-line program for the data-prep-to-quick-estimate loop economists use Stata for: import, clean, generate variables, run regressions, panel methods, MLE, IV, and ARIMA — plus publication-grade SVG graphics and six bundled practice datasets, including the complete IMF World Economic Outlook database. The syntax is close to Stata’s where they overlap; the semantics are faithful where it matters (missing-value algebra, the `by:` prefix, gap-aware time-series operators, factor variables). `tea` also runs entirely in a web browser via WebAssembly, with no installation, and its output is byte-identical across machines and numerical libraries. This manual covers installation, a guided tour, every command, the bundled data, and escape hatches to Python/R for things `tea` does not handle.

Audience. Applied economists, policy analysts, and graduate students familiar with Stata. No Stata experience is strictly required, but the manual assumes you know what `regress` is and what panel data looks like.

Status. v1.2.2 follows the v1.0 field-testing release. It has passed 42 regression tests byte-identically on three configurations (native gcc + OpenBLAS in release and sanitizer builds, and WebAssembly clang + reference BLAS), is clean under AddressSanitizer and UndefinedBehaviorSanitizer, and builds warning-free under `-Wall -Wextra -Werror`. Field testing continues; please report what breaks.

AI-assisted development. This software was developed with the assistance of an AI coding assistant (Anthropic’s Claude). The design decisions, mathematical specifications, scope choices, and final acceptance of each component were the author’s; the AI contributed substantial portions of the implementation, drafted regression tests, and produced the first draft of this manual. All code was reviewed before inclusion. This manual was likewise drafted with AI assistance and edited by the author. Bugs and design flaws are the author’s responsibility.

License. GPLv3 or later. Copyright © 2026 Mico Mrkaic. See the `LICENSE` file in the distribution for the full text. `tea` is an independent open-source project, not affiliated with or endorsed by StataCorp LLC. The bundled datasets retain their original licenses and citations; see the “Bundled datasets” chapter and `data/SOURCES.md`.

Contents

tea — user guide	4
1. What tea is	4
2. Getting tea	5
3. A first session	5
4. Core concepts	6
5. Data management	7
6. Time series and panels	8
7. Estimation	8
8. Graphics	9
9. Bundled datasets (sysuse)	9
10. Randomness and reproducibility	10
11. The browser edition	10
12. Programming	10
13. When to leave tea	11
14. Troubleshooting and reporting	11
 Command reference	 11
scatter	11
line	11
histogram	11
generate	12
replace	12
egen	12
list	12
summarize	12
tabstat	12
count	12
describe	12
drop	12
keep	12
rename	13
order	13
aorder	13
label	13
format	13
sort	13
gsort	13
tabulate	13
tsset	13
xtset	13
xtdescribe	13
xtreg	14
hausman	14
logit	14
probit	14
ivregress	14
poisson	14
arima	14
margins	14
estimates	14
est	15

estout	15
collapse	15
merge	15
reshape	15
recode	15
encode	15
decode	15
destring	15
tostring	15
codebook	15
import	16
export	16
save	16
sysuse	16
use	16
clear	16
frame	16
regress	16
predict	16
test	16
lincom	16
help	17
pwd	17
cd	17
mkdir	17
dir	17
rmdir	17
erase	17
copy	17
do	17
version	17
log	17
exit	17
Native statements	17

tea — user guide

This is the master Markdown source of the tea manual. The PDF is generated from it (`make manual`), and the command reference appendix is generated from the tea binary itself — so neither can silently drift from the implementation.

1. What tea is

`tea` (*tiny econometric assistant*) covers the slice of Stata that applied economists actually use day to day: import data, clean it, generate variables, reshape, run regressions and panel estimators, and produce publication-ready tables and figures. It does not try to be all of Stata. Scope discipline is a feature: the program is one small, fast-starting binary with no license server, no runtime, and no installation beyond copying a file.

Three design commitments shape everything else:

Faithful semantics where it matters. Missing-value algebra follows Stata exactly, including the intentional footgun that `if x > 5` *includes* missing values (missing sorts above every number). The `by`:

prefix errors on unsorted data, exactly like Stata. Time-series operators are gap-aware. If you have Stata muscle memory, it should transfer — and where tea deliberately differs, the difference is documented in COMPATIBILITY.md.

One binary, everywhere. The same program builds on Linux and macOS, and compiles to WebAssembly so it runs in a browser tab with nothing installed (see chapter 11). The six bundled practice datasets — including the complete IMF World Economic Outlook — live *inside* the binary, so a single file is a complete practice environment.

Reproducible output. The same do-file produces byte-identical output on every machine, operating system, CPU, and numerical library. This is a documented promise with specific engineering behind it (chapter 10), verified by running the full regression suite on maximally different numerical backends.

2. Getting tea

The browser edition (zero install)

Open <https://micomrkaic.github.io/tea/> in any modern browser. The full program — estimators, plots, bundled data — runs locally in the tab via WebAssembly. No data you load or upload leaves your machine.

Building from source

```
git clone https://github.com/micomrkaic/tea
cd tea
make check-deps      # tells you which -dev packages are missing
make                 # produces ./tea
make test            # 42/42 expected
```

Dependencies (Debian/Ubuntu):

```
apt install libreadline-dev libopenblas-dev liblapacke-dev \
            libgsl-dev libreadstat-dev
```

macOS: brew install readline openblas lapack gsl readstat.

Copy ./tea anywhere on your PATH. That is the whole installation.

Running

```
tea                # interactive REPL
tea mywork.do      # run a do-file (batch)
tea -q            # REPL without the startup banner
tea --tea-extensions # enable extensions beyond strict Stata behavior
tea --version
```

3. A first session

Start tea and try the thirty-second tour on bundled data:

```
. sysuse weo
(weo: 10168 obs loaded - IMF World Economic Outlook, April 2026: ...)
. keep if aggregate==0          // countries only; ==1 keeps World, G7, ...
. xtset iso year
. xtreg ngdp_rpch ggxdg_ngdp if year<=2025, fe
. scatter pcpipch lur if year==2024, title("Inflation vs unemployment, 2024")
```

That is a growth-on-debt fixed-effects regression across ~190 economies and a cross-country scatter, on the real World Economic Outlook, in five lines. `sysuse dir` lists all six bundled datasets (chapter 9).

The REPL offers readline editing: arrow keys, history (Up/Down), Ctrl-A/E/K/U/W, Ctrl-L to clear, and Tab completion of command names, variable names, and `sysuse` dataset names. The browser edition has the same editing (chapter 11).

4. Core concepts

The dataset in memory

Like Stata, tea holds one active dataset in memory: a rectangle of observations by variables. Commands that would discard unsaved data (`use`, `sysuse`, `import`, `clear`) refuse unless you add `, clear`. Multiple datasets can be held simultaneously via **frames** (`frame create/change/copy/put/drop/dir`); each frame has its own sort and panel state.

Variables are `double` or `string`. Stata's `byte/int/long/float` types are accepted in input files but stored as `double` — a deliberate simplification that costs a little memory and removes a class of precision surprises.

Missing values

Numeric missing is written `.` and behaves exactly as in Stata:

- any arithmetic with missing yields missing;
- missing compares **greater than every number**, so `keep if x > 5` keeps missing-`x` rows — write `keep if x > 5 & !missing(x)` when that is not what you mean;
- `missing(x)` and `!missing(x)` are the idiomatic tests.

This footgun is preserved *because* it is Stata's: code and habits transfer unchanged.

Expressions

Anywhere an expression is accepted (`generate`, `replace`, `if` qualifiers, `assert`, `display`) you get arithmetic, comparisons, `&` `|` `!`, string literals, subscripts (`x[_n-1]`), the builtins `_n`, `_N`, `_pi`, and a function library (`ln`, `exp`, `sqrt`, `abs`, `mod`, `round`, `floor`, `ceil`, `min`, `max`, `cond`, `missing`, `runiform`, `rnormal`, string functions, and more — see the reference).

A variable name that does not exist is an **error** (`r(111)`), caught before anything is evaluated — a typo in `drop if` can never silently destroy data.

Do-files, comments, continuation

```
* full-line comment
regress y x           // trailing comment
regress y ///
    x1 x2 x3          // /// continues the line
#delimit ;
regress y x1 x2 ; #delimit cr is implied by a plain "#delimit"
```

Batch mode (`tea file.do`) echoes commands, aborts on the first error with the offending line number, and never opens plot viewers — output is fully deterministic, which is what makes golden-file testing (and the reproducibility promise) possible.

`log using results.log` tees subsequent output to a file; `log close` stops.

Getting help

`help` lists every command; `help CMD` prints the same text reproduced in this manual's reference (Part II) — the manual's reference is *generated from* those help strings, so the two cannot disagree.

5. Data management

Loading and saving

```
use mydata.dta [, clear]           // Stata .dta via readstat
save out.dta [, replace]
import delimited file.csv [, clear delimiters(";")]
import excel file.xlsx [, clear]   // needs ssconvert or libreoffice
export delimited out.csv [, replace]
sysuse NAME [, clear]             // bundled data, chapter 9
```

CSV import auto-detects types; `destring/tostring` convert after the fact; `encode/decode` map between string categories and labeled numerics.

Creating and transforming variables

```
generate lgdp = ln(gdp)
replace lgdp = . if gdp <= 0
egen      mgdp = mean(gdp), by(country)    // group statistics
egen      gid  = group(country)           // 1..G group index
recode    x (1/5 = 1) (6/10 = 2), gen(xcat)
```

`egen` functions include mean, sum, sd, min, max, count, median, percentiles, group, and row-wise variants — `help egen` lists the exact set.

Selecting, ordering, labeling

```
keep if year >= 2000
drop if missing(gdp)
keep country year gdp           // variable selection
sort country year
gsort -gdp                      // descending
by country: generate t = _n      // by: requires sorted data
bysort country (year): gen dgdp = gdp - gdp[_n-1]
rename oldname newname
order id country year
label variable gdp "GDP, constant prices"
format gdp %12.2f
```

`by varlist:` is faithful: it errors if the data is not sorted on `varlist` (use `bysort` to sort first), and `by g (s):` groups by `g` sorting within by `s`. Under `by:`, `_n` and `_N` are within-group.

Restructuring

```
reshape long gdp, i(country) j(year)    // wide -> long (in place)
reshape wide gdp, i(country) j(year)
merge 1:1 id using other.dta            // also m:1; m:m is refused
collapse (mean) gdp inflation, by(region year)
append is not implemented - see COMPATIBILITY.md
```

`reshape` operates in place; `frame copy` first if you want to keep the original. `merge m:m` is deliberately excluded as a footgun.

6. Time series and panels

```
tsset year                // pure time series
xtset country year        // panel: country id (numeric or string) + time
xtdescribe                // pattern summary
```

Once set, the gap-aware operators work in any expression and any estimator's varlist:

```
L.x  L2.x    lags          F.x  F2.x    leads
D.x  D2.x    differences   S4.x          seasonal difference
```

Gap-aware means L.x at time t looks up $t-1$ in the panel — if that period is absent, the result is missing rather than the previous row's value. Order-breaking commands (`sort` on other keys, `drop` of the panel variable, ...) invalidate the settings, and tea tells you.

Factor variables expand in estimation varlists:

```
regress y i.region        // region dummies (first level base)
regress y c.x##i.region    // interactions, Stata syntax
```

7. Estimation

All estimators share the `[if]` `[in]` `[weight]` machinery, print Stata-style tables, and leave results in `_b[name]` / `_se[name]` and estimates storage.

Linear regression

```
regress y x1 x2 [aw=w]          // classical / analytic weights
regress y x1 x2, robust         // HC1
regress y x1 x2, cluster(country) // cluster-robust
regress y L.y D.x i.region      // TS-ops and factors in varlists
```

`fweight`, `aweight`, `pweight`, `iweight` are supported where they make sense; `pweight` implies robust standard errors.

Instrumental variables

```
ivregress 2sls y (x = z1 z2) w1 w2
```

Panel estimators

```
xtreg y x, fe                // within estimator; u_i F-test
xtreg y x, re                // GLS random effects (theta reported)
xtreg y x, be                // between estimator
hausman                      // FE vs RE, after running both
```

Limited dependent variables and counts

```
logit y x1 x2                // MLE with Newton iterations shown
probit y x1 x2
poisson visits chronic school income
margins, dydx(x1)            // average marginal effects
```

Time series

```
arima y, arima(1 0 1)        // conditional-likelihood ARIMA(p,d,q)
```


After estimation

```
predict yhat           // linear prediction (xb)
predict e, resid       // residuals; xtreg adds u / e / ue / xbu
test x1 = x2           // Wald tests
lincom x1 + 2*x2
estimates store m1
estimates table m1 m2, se
estout m1 m2 using table.tex // publication tables (LaTeX/TSV)
```

Perfect fits and degenerate cases

When a regression fits exactly, tea reports residual SS 0, SEs 0, $t = 0.00$, $p = 1.000$, and $F = \text{inf}$ — deterministically, on every machine. See chapter 10 for why this is a design decision and not an accident.

8. Graphics

Three commands cover the everyday cases, rendered by tea’s own dependency-free SVG engine:

```
scatter y x    [if] [in] [, title() xtitle() ytitle() saving(FILE) noview]
line y x      [if] [in] [, sort ...same options...]
histogram v   [if] [in] [, bins(#) freq ...same options...]
```

- Output is a vector SVG — publication quality by default, and identical byte-for-byte across platforms (plots are covered by golden-file tests).
- In the interactive REPL the plot opens in your OS viewer; `saving(FILE)` writes to a named file instead; `noview` suppresses the viewer.
- In do-files plots are only ever written to files, never opened.
- `line` connects points in data order; add `sort` to order by `x`.
- `histogram` shows density by default; `freq` for counts; automatic binning is `min(ceil(sqrt(N)), 50)`.
- In the browser edition plots appear in the panel beside the terminal.

9. Bundled datasets (sysuse)

Six practice datasets are embedded inside the binary — no files, no network, identical in the browser:

```
. sysuse dir
grunfeld  Grunfeld (1958) investment panel: 10 US firms x 1935-1954 (xtreg, hausman)
airline   Box-Jenkins airline passengers, monthly 1949-1960 (tsset, arima)
longley   Longley (1967) US macro, 16 obs: famously ill-conditioned (regress)
nmes1988  Deb-Trivedi (1997) medical care demand, 4406 persons 66+ (poisson, logit)
pwt       Penn World Table 10.0 sample: 22 countries x 1950-2019, CC BY 4.0 (growth)
weo       IMF World Economic Outlook, April 2026: 197 countries + 13 aggregates, 145 indicators
```

Load with `sysuse NAME[, clear]`. Highlights:

- **weo** is the complete published WEO database, April 2026 vintage: long panel `country iso year` `aggregate` plus one variable per WEO subject code, lowercased (`ngdp_rpch` real GDP growth, `pcpipch` CPI inflation, `lur` unemployment, `bca_ngdpd` current account %GDP, `ggxwdg_ngdp` gross debt %GDP, ...). World and regional groups (World, G7, Euro Area, EU, EMDEs, ...) carry `aggregate==1`; countries `aggregate==0`. 101 of the 145 codes are published for groups only and are empty on country rows. The full code → descriptor → units table is in `data/weo_codes.txt`. Years through 2031 are the projections *as published in this vintage* — a practice snapshot, not a live data service. Cite as *IMF, World Economic Outlook Database, April 2026*.

- **grunfeld** reproduces the textbook FE estimates (0.110, 0.310); **longley** reproduces the certified solution of a famously ill-conditioned problem — a good stress test of any least-squares code.
- Provenance, licenses, and citations for all six: `data/SOURCES.md`. To regenerate the embedded data (e.g. a newer WEO vintage): `tools/curate_weo.py` then `tools/gen_sysdata.py` and rebuild.

10. Randomness and reproducibility

Random numbers

`runiform()` and `rnormal(mean, sd)` draw from a PCG32 generator; `set seed N` makes every subsequent draw reproducible. The stream is identical across platforms.

Backend-independent output

tea promises that **the same do-file prints byte-identical output on every machine, BLAS library, and CPU**. Floating-point results of well-posed computations already agree to displayed precision everywhere; the danger zone is *mathematical zeros* — the residual sum of squares of a perfect fit, the SE of an exactly-determined coefficient — which different numerical backends render as different sub-epsilon noise (5.9e-29 here, 4.1e-29 there), occasionally flipping what gets printed entirely.

tea therefore snaps such quantities to exactly zero when they are below tight relative tolerances (residual SS under 1e-12 of total SS, and the consequences that follow: residual vectors, SEs, coefficients under 1e-8 of the largest in an exact fit, `sigma_u` for identical panel intercepts, the `hausman` difference matrix at machine precision). A perfect fit reports `F = inf`, `p = 0.0000`. **Normal estimation results are untouched to the last bit** — the thresholds only fire on genuine degeneracy.

This is verified, not asserted: the full regression suite passes byte-identically under native gcc + OpenBLAS and under WebAssembly clang + reference BLAS + musl — two maximally different numerical stacks.

11. The browser edition

The WebAssembly build at <https://micomrkaic.github.io/tea/> is the same program: same commands, same estimators, same bundled data, same output to the last byte (the regression suite runs inside the browser build too). Practical differences:

- **Files** live in an in-browser filesystem. *Upload data file* places files where `use/import delimited` can see them; *Download workspace files* retrieves anything you `save/export`. Nothing leaves your machine — there is no server side.
- **Plots** render in the panel beside the terminal instead of an OS viewer.
- **shell is unavailable** (browsers have no processes) and fails with a clear message.
- The terminal offers the same readline-style editing and Tab completion as the native REPL.

The first load fetches ~2 MB (compressed); afterwards the browser caches it.

12. Programming

```
local rhs "x1 x2 x3"
global outdir "results"
display `rhs' // macro expansion with ` '
foreach v in gdp inflation unemp {
    summarize `v'
}
forvalues i = 1/10 {
    quietly replace x = x + `i' in `i'
}
while `k' < 10 { ... }
```

```

if `cond' { ... } else { ... }
capture noisy-command           // swallow errors (note: _rc not yet wired)
quietly regress y x             // suppress output; results still stored
program define myprog
    ...
end
assert _N == 42                 // fail loudly if untrue

```

Command-level `if` evaluates macros and scalars; unlike Stata it does not resolve variable names to first-observation values (it errors instead — see `COMPATIBILITY.md`).

13. When to leave tea

tea is deliberately small. For anything outside its scope — heavy graphics grammars, Bayesian estimation, mixed models, big-data joins — the escape hatch is one line:

```

export delimited handoff.csv
shell python3 analyze.py handoff.csv

```

`shell` (or `!cmd`) runs anything on your system; tea orchestrates, the system provides. The `README` lists the escape hatches in more detail.

14. Troubleshooting and reporting

- `make check-deps` diagnoses missing build dependencies with the exact package names.
- A typo'd variable name anywhere in an expression is error `r(111)`; your data is never modified by a failed command.
- Batch runs abort at the first error with `do-file aborted at line N`.
- Known issues live in `KNOWN_BUGS.md`; current entries include `capture` not suppressing error messages, `_rc` not yet carrying return codes, and an `arima` convergence question on differenced series.
- Found something new? That is the point of this release — open an issue at github.com/micomrkaic/tea with the command, what you expected, and what happened. A three-line reproduction is worth more than a page of description.

Command reference

Generated from the tea binary by `tools/gen_cmdref.sh` — this text is exactly what `help CMD` prints at the prompt, and regenerating it after any change keeps the manual and the implementation in agreement.

scatter

```

scatter yvar xvar [if] [in] [, title() xtitle() ytitle() saving() noview]
    SVG scatter plot; saving(FILE) writes to FILE instead of tea_graph.svg
    e.g.  scatter gdp_growth inflation if year>2000, title("Growth vs inflation")

```

line

```

line yvar xvar [if] [in] [, sort title() xtitle() ytitle() saving() noview]
    SVG line plot; connects points in data order (use sort to order by x)
    e.g.  line gdp year if country=="US", sort

```

histogram

```

histogram var [if] [in] [, bins(#) freq title() saving() noview]

```

SVG histogram; density by default, freq for counts, auto bins = min(ceil(sqrt(N)),50)
 e.g. histogram wage, bins(30) freq

generate

generate [type] newvar = exp [if] [in] create a new variable
 e.g. gen logy = log(income) if year >= 2000

replace

replace var = exp [if] [in] modify existing variable
 e.g. replace income = . if income < 0

egen

egen newvar = fn(args) [, by(varlist)] extended generate (mean/sum/group/...)
 e.g. egen meanY = mean(y), by(country)

list

list [varlist] [if] [in] print observations
 e.g. list country year gdp if year>2020 in 1/10

summarize

summarize [varlist] [if] [in] [weight] [, detail] N/mean/sd/min/max; sets r()
 e.g. summarize gdp_growth, detail

tabstat

tabstat varlist [if] [in], statistics(stat ...) [by(var)] [columns(stat|var)]
 stats: mean sd var cv min max range sum count median p1-p99 iqr
 e.g. tabstat gdp pop, stats(mean sd p25 p50 p75) by(region)

count

count [if] [in] [fw=] count matching observations

describe

describe [varlist] show variables, types, formats

drop

drop varlist | if exp | in range remove vars or observations
 e.g. drop if gdp == . | drop tempvar1 tempvar2

keep

keep varlist | if exp | in range complement of drop
 e.g. keep country year gdp

rename

rename oldname newname rename a variable
e.g. rename _C* C* (wildcard rename: strip leading underscore)

order

order varlist reorder variables (listed ones go first)
e.g. order country year

aorder

aorder [varlist] alphabetize variable order

label

label variable v "text" attach a description to a variable
label define lblname # "text" # "text" define a value-label set
label values varlist lblname attach value labels to variables

format

format varlist %fmt set display format
e.g. format date %td | format gdp %12.2f

sort

sort varlist ascending stable sort
e.g. sort country year

gsort

gsort [+/-]var [+/-]var ... sort with per-key direction
e.g. gsort -gdp +country (gdp desc, then country asc)

tabulate

tabulate var [if] [in] [weight] one-way frequency table
e.g. tabulate region if year == 2020

tsset

tsset timevar [, format(%fmt)] declare time series
e.g. tsset year

xtset

xtset panelvar timevar declare panel (gap-aware L./F./D./S.)
e.g. xtset country year

xtdescribe

xtdescribe summarize panel structure
e.g. xtdescribe (requires xtset first; shows n, T, balance)

xtreg

```
xtreg y x1 x2 ... [if] [in], fe [vce(robust|cluster v)]    panel FE OLS
e.g.  xtreg growth L.growth pop, fe
      (requires xtset; within estimator; reports R-w/b/o, sigma_u/e, rho)
```

hausman

```
hausman                                FE vs RE specification test
e.g.  xtreg y x, fe
      xtreg y x, re
      hausman    (Ho: cov(u_i, x) = 0; rejection means use FE)
```

logit

```
logit y x1 x2 ... [if] [in] [weight], [vce(robust|cluster v)]    logistic regression
e.g.  logit voted income education, vce(cluster county)
```

probit

```
probit y x1 x2 ... [if] [in] [weight], [vce(robust|cluster v)]    probit regression
e.g.  probit voted income education, vce(robust)
```

ivregress

```
ivregress 2sls y [exog] (endog = instruments) [if] [in] [weight] [, vce(robust|cluster v)]
Two-stage least squares with first-stage F diagnostic.
e.g.  ivregress 2sls wage educ (exper = age momeduc daddyEduc)
```

poisson

```
poisson y x1 x2 ... [if] [in] [weight] [, vce(robust|cluster v)]    poisson regression
e.g.  poisson accidents speed_limit population, vce(cluster state)
```

arima

```
arima y [exog] [if] [in], arima(p d q) [noconstant]    ARIMA(p,d,q) via conditional ML
e.g.  arima gdp, arima(1 1 1)                          ARIMA(1,1,1) on gdp
      arima cpi unemp, arima(2 0 1)                    ARMAX(2,1) with exog regressor
```

margins

```
margins , dydx(*|varlist) [atmeans]                    average marginal effects
e.g.  logit y x1 x2
      margins, dydx(*)                                AME for all regressors
      margins, dydx(x1) atmeans                        MEM for x1 at sample means
```

estimates

```
estimates store|restore|dir|drop|table                named estimates ledger
e.g.  regress y x1 x2
      estimates store m1
      regress y x1 x2 x3
      estimates store m2
      estimates table m1 m2, se star    stats(N r2 rmse)
```

est

est = estimates (short form) see ``help estimates``

estout

estout [names] [, format(latex|markdown|plain) se|t|p stars stats(...)]
Side-by-side LaTeX/markdown/plain table of stored estimates.
e.g. estout m1 m2, stats(N r2 rmse) using table.tex

collapse

collapse (stat) v1 v2 ... , by(g) [weight] aggregate to groups
e.g. collapse (mean) gdp pop, by(region year)

merge

merge 1:1|m:1|1:m keyvars using FILE [, ...] join master with using file
e.g. merge 1:1 country year using gdp.tea (m:m is rejected by design)

reshape

reshape long|wide stubs , i(idvars) j(jvar) pivot wide<->long (in-place)
e.g. reshape long v, i(country) j(year)
takes v1980 v1981 ... v2031 -> rows of (country, year, v)

recode

recode varlist (rules) [, gen(stub)] map values
e.g. recode score (1/3=1)(4/6=2)(7/10=3) (missing=.), gen(score_g)

encode

encode strvar, generate(newvar) [label(name)] map strings to integers + value label

decode

decode intvar, generate(newvar) inverse of encode (codes back to strings)

destring

destring varlist, {replace|generate(stub)} [force] [ignore("chars")] str -> num
e.g. destring z4, replace force (junk values become missing)

tostring

tostring varlist, {replace|generate(stub)} [force] [format(%fmt)] num -> str

codebook

codebook [varlist] type, uniques, missing, range

import

```
import delimited|excel|ods using FILE [, sheet(name) clear]    read CSV/TSV/XLSX/ODS
e.g. import delimited "data/WE0.csv", clear
```

export

```
export delimited using FILE                                write CSV/TSV
e.g. export delimited using out.csv
```

save

```
save FILE [, replace]                                write Stata .dta (default) or .tea
e.g. save mydata.dta, replace                        - emits Stata-compatible .dta
e.g. save mydata.tea, replace                        - native tea binary
```

sysuse

```
sysuse dir | sysuse NAME [, clear]
load a practice dataset bundled inside the tea binary
e.g. sysuse grunfeld, clear
xtset firm year
xtreg invest value capital, fe
```

use

```
use FILE [, clear]                                read Stata .dta, .tea, .csv, or .tsv
e.g. use mydata.dta, clear                        - extension dispatch decides format
'use foo' with no extension looks for foo.dta (Stata default).
```

clear

```
clear                                                drop all data in current frame
```

frame

```
frame create|change|copy|rename|put|drop|dir multiple datasets
e.g. frame create alt | frame change alt | frame put x y, into(alt)
```

regress

```
regress y x1 x2 ... [if] [in] [weight] [, noconstant robust cluster(var)] OLS
e.g. regress gdp_growth investment trade if year>=2000, cluster(country)
```

predict

```
predict newvar [, xb residuals]                    predict from last fit
```

test

```
test v1 v2 ...                                    Wald F-test (joint zero)
```

lincom

```
lincom <linear combo of coefs>                    point estimate + SE of L'b
```


help

```
help [cmd]                list commands or show one's syntax
```

pwd

```
pwd                                print working directory
```

cd

cd DIR	change working directory
--------	--------------------------

mkdir

```
mkdir DIR [, recursive]           create directory (recursive = mkdir -p)
```

dir

```
dir [pattern]                list files in current directory
```

```
rmmdir
```

```
rmmdir DIR                remove empty directory
```

erase

```
erase FILE | rm FILE           delete a file
```

copy

```
copy SRC DST [, replace]          copy a file
```

do

```
do FILENAME                                run another do-file
```

version

```
version                                tea version & build info
```

log

```
log using FILE [, replace append] | log close    tee output to a file
```

exit

```
exit [, clear]           leave tea (clear drops data first)
```

Native statements

Handled by the interpreter before command dispatch: `display`, `assert`, `shell` (and the `!cmd` escape), `#delimit`, `local`, `global`, `foreach`, `forvalues`, `while`, `if/else`, `capture`, `quietly`, `program` `define`, and comments (`*`, `//`, `///`).